Practical 1: Write a program to implement Breadth-First Search (BFS) for graph traversal.

```
from collections import deque
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}
def bfs(graph, start, goal):
    visited = set()
    queue = deque([start])
    visited.add(start)
    while queue:
        node = queue.popleft()
        print("Visited:", node)
        if node == goal:
            print("\nGoal node", goal, "found!")
            return
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    print("\nGoal node", goal, "not found!")
start = 'A'
goal = 'D'
bfs(graph, start, goal)
```

‘’’Output

Visited: A

Visited: B

Visited: C

Visited: D

Goal node D found!

‘’’

Practical 2: Write a program to implement Depth-First Search (DFS) for graph traversal.

```
def dfs(graph,start,goal):
    visited = []
    stack= []
    stack.append(start)
    visited.append(start)
    print('The path traversed is:')
    while stack:
        element=stack.pop()
        print(element," ")
        if(element==goal):
            break
        for neighbor in graph[element]:
            if neighbor not in visited:
                stack.append(neighbor)
                visited.append(neighbor)
graph={
    'A':['C','B'],
    'B':['E','D'],
    'C':['G','F'],
    'D':[],
    'E':['I','H'],
    'F':[],
    'G':['J'],
    'H':[],
    'I':[],
    'J':[]
```

```
}
start='A'
goal='J'
dfs(graph,start,goal)

'''
The path traversed is:
A
B
D
E
H
I
C
F
G
J

'''
```

3 Practical Name : Write a program to solve the 8-puzzel problem using a state-space search approach

```
From collections import deque
Def bfs(start,goal):
Queue = deque([(start,[])])
   Visited=set()
   While queue:
  State,path=queue.popleft()
   If state==goal:
Return path+[state]
 If state in visited:
 Continue
      Visited.add(state)
       Zero=state.index(0)
       Row,col=divmod(zero,3)
       For dr,dc in[(-1,0),(1,0),(0,-1),(0,1)]:
             R,c=row+dr,col+dc
          If 0<=r<3 and 0<=c<3:
               New=list(state)
              New[zero],new[r*3+c]=new[r*3+c],new[zero]
              Queue.append((tuple(new),path+[state]))
       Return None
Start = (1, 2, 3,4, 0, 6,7, 5, 8)
Goal = (1, 2, 3,4, 5, 6,7, 8, 0)
Solution=bfs(start,goal)
Print("Solution steps")
For I,state in enumerate(solution):
   Print("Step",i)
   Print(state[:3])
   Print(state[3:6])
   Print(state[6:])
   Print()
```

OUTPUT:
Solution steps

Step 0
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

Step 1
(1, 2, 3)
(4, 5, 6)
(7, 0, 8).

Step 2.

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

4 Practical Name : Write a program to solve the Monkey and Banana Problem using a sequence of action.

```
State=(“door”,”floor”,”banana”)
Goal=(“box”,”on_box”,”banana_taken”)
Steps=[]
If state[0]!=”box”:
  Steps.append(“Monkey moves to the box”)
  State=(“box”,”floor”,”banana”)
If state[0]==”box”:
  Steps.append(“Monkey pushes the box under the banana”)
  State=(“banana”,”floor”,”banana”)
If state[0]==”banana”:
  Steps.append(“Monkey climb on the box”)
  State=(“banana”,”on_box”,”banana”)
If state[1]==”on_box”:
  Steps.append(“Monkey takes banana”)
   State=(“banana”,”on_box”,”banana”)
Print(“Monkey & Banana Problem”)
Print(“……..”)
For I,step in enumerate(steps,1):
    Print(“step”,I,”:”,step)
Print (“\nGoal state Reached !”)
Print(“Monkey has the banana”)
```

OUTPUT:

Monkey & Banana Problem

…….

Step 1 : Monkey moves to the box

Step 2 : Monkey pushes the box under the banana

Step 3 : Monkey climb on the box

Step 4 : Monkey takes banana

Goal state Reached !

Monkey has the banana ‘’’

5 Practical Name : Write a program to implement the Hill Climbing Algorithm.

```
Def simple_hill_climbing(numbers):
   Current_index = 0
   While True:
     If current_index + 1 < len(numbers):
        If numbers[current_index] < numbers[current_index + 1]:
           Current_index += 1
        Else:
           Return numbers[current_index]
      Else:
        Return numbers[current_index]
Numbers = [6, 14, 21, 34, 47, 67]

Max_number = simple_hill_climbing(numbers)

Print(f”The maximum number in the list is: {max_number}”)
```

OUTPUT:
The maximum number in the list is: 67

Practical 6
Implementation of Informed Search Technique : Best First Search

```
From queue import PriorityQueue
Def best(graph,start,goal,heuristic):
   Queue=PriorityQueue()
   Queue.put((heuristic[start],start))
   Visited=list()
   While not queue.empty():
      hValueCurrent,current=queue.get()
      visited.append(current)
      print(current," ",hValueCurrent)
      if current == goal:
         return True
      for node in graph[current]:
         queue.put((heuristic[node],node))
   return False
graph={
   'A':['B','C'],
   'B':['D','E'],
   'C':['F'],
   'D':[],
   'E':['G'],
   'F':[],
   'G':[]
}
Heuristic={
   'A':10,
   'B':5,
   'C':7,
   'D':3,
   'E':5,
   'F':8,
   'G':9
}
Print("Goal Found") if best(graph,'A','J',heuristic) else print("Goal Not Found")
```

OUTPUT :

```
A  10
B  5
D  3
E  5
C  7
F  8
Goal Found
```

Practical 7Implementation of A* Searching Technique

```
Def aStar(graph,heuristic,start,goal):
   If start==goal:
     Print("Path Found\nPath : ",start)
      Return
   Cost=dict()
   Cost[start]=0
   Parent[start]=None
   Mi=start
   While cost:
      If graph[mi]:
        For node in graph[mi]:
            If node[0] not in graph.keys():
               Continue
            Parent[node[0]]=mi
            Cost[node[0]]=(cost[mi]-heuristic[mi])+node[1]+heuristic[node[0]]
      Cost.pop(mi)
      Mi=None
      For I in cost.keys():
         If mi==None or cost[i]<cost[mi]:
            Mi=i
      If mi==goal:
         Print("Path Found")
         Path=[mi]
         Path(mi,path)
         Return
   Print("Path Not Found")
Def Path(node,path):
   If parent[node]==None:
      Path.reverse()
      Print("Path : ",*path)
      Return
   Path.append(parent[node])
   Path(parent[node],path)
Parent=dict()
Graph={
   'A':[('B',2),('E',3)],
    'B': [('C',1),('G',7)],
    'C':None,
    'D':[('G',1)],
    'E':[('D',6)],
    'G':None
 }
Heuristic={
   'A':11,
   'B':6,
   'C':99,
   'D':1,
   'E':7,
   'G':0
}
aStar(graph,heuristic,'A','G')
```

Practical 8: Implementation of AO* Searching Techinque

```
Class Graph:
   Solution=dict()
   Def __init__(self,graph,heuristic):
      Self.graph=graph
      Self.heuristic=heuristic
   Def h(self,node):
      Return self.heuristic[node]
   Def AO(self,start):
      If start==None:
         Return
      For n in start:
         groupCost=dict()
         for group in self.graph[n]:
            cost=0
            key=""
            for node in group:
               cost+=node[1]+self.h(node[0])
               key=key+node[0]
            groupCost[key]=cost
         mi=None
         for group in groupCost.keys():
            if mi==None or groupCost[group]<groupCost[mi]:
               mi=group
         if mi!=None:
            self.solution[n]=list(map(lambda a:a,mi))
         else :
            self.solution[n]=mi
        print(f"\nProcessing Node {n}\nSelected Path : {mi if mi==None else list(map(lambda a:a,mi))}")
         self.AO(mi)
   def Solution(self):
      print("\nFinal Solution : \n")
      for node in self.solution.keys():
         print(f"{node} : {self.solution[node]}")
g={
   'A':[[('B',1),('C',1)],[('D',1)]],
   'B':[[('G',1)],[('H',1)]],
   'C':[[('J',1)]],
   'D':[[('E',1),('F',1)]],
   'E':[],
   'F':[],
   'G':[[('I',1)]],
   'H':[],
   'I':[],
   'J':[]
}
He={
   'A':1,
   'B':6,
   'C':2,
```

```
    'D':12,
    'E':2,
    'F':1,

    'G':5,
    'H':7,
    'I':7,
    'J':1
}
Graph=Graph(g,he)
Graph.AO('A')
Graph.Solution()
```

OUTPUT :

```
Processing Node A
Selected Path : ['B', 'C']

Processing Node B
Selected Path : ['G']



Processing Node G
Selected Path : ['I']

Processing Node I
Selected Path : None

Processing Node C
Selected Path : ['J']

Processing Node J
Selected Path : None



Final Solution :



A : ['B', 'C']

B : ['G']

G : ['I']

I : None

C : ['J']

J : None
```

Practical 9 : Write a program to Implement a simple knowledge base using facts and rules.

```
Facts = {
  “ram_is_human”: True,
   “shyam_is_human”: True,
   “ram_is_male”: True,
   “shyam_is_male”: True
}
Def knowledge_base():
   If facts[“ram_is_human”]:
      Print(“Ram is a human.”)
   If facts[“shyam_is_human”]:
      Print(“Shyam is a human.”)
   If facts[“ram_is_human”] and facts[“ram_is_male”]:
      Print(“Ram is a man.”)
   If facts[“shyam_is_human”] and facts[“shyam_is_male”]:
      Print(“Shyam is a man.”)
Knowledge_base()
```

OUTPUT

Ram is a human.

Shyam is a human.

Ram is a man.

Shyam is a man.

Practical 10 : Write a program to Implement Forward Chaining for rule-based inference.

```
Facts = [“rain”]
If “rain” in facts:
   Facts.append(“cloudy”)
If “cloudy” in facts:
   Facts.append(“carry_umbrella”)
Print(“Facts:”, facts)

#Output

Facts: [‘rain’, ‘cloudy’, ‘carry_umbrella’]
```

Practical 11 : Write a program to Implement Backward Chaining for goal-based reasoning.

```
Facts = [“rain”]
Goal = “carry_umbrella”
Rules = {
   “cloudy”: [“rain”],
   “carry_umbrella”: [“cloudy”]
}
Def backward_chaining(goal):
   If goal in facts:
      Return True
   If goal in rules:
      For condition in rules[goal]:
         If backward_chaining(condition):
            Return Tru
   Return False
If backward_chaining(goal):
   Print(“Goal Achieved:”, goal)
Else:
   Print(“Goal Not Achieved”)
```

#OUTPUT

Goal Achieved: carry_umbrella